Table 4. Overview of qualitative downstream task benchmarks

Benchmark	Type	Used Split	Split Sample Size
HellaSwag	Commonsense Reasoning	validation	10042
Winogrande	Commonsense Reasoning	validation	1267
ARC easy	Question Answering	test	2376
ARC challenge	Question Answering	test	1171
TruthfulQA	Question Answering	test	817

special characters causing failures in llama-perplexity. For HellaSwag and Winogrande, the validation dataset splits were used since only unlabeled test splits were available. The Flan T5 Large model was excluded from the qualitative assessment due to compatibility issues with llama-perplexity. All qualitative benchmarks were conducted in a zero-shot setting (i.e., without model fine-tuning on these datasets) using a context size of 512.

5 Evaluation Results

This section presents the experimental results. As described earlier, five runs were conducted for each model and configuration, with only runs two to five reported, as the first run served as a warm-up phase. However, differences between the first run and subsequent runs were minimal, except during the load phase, where extended loading times were observed. This phase refers to llama.cpp initializing and loading the model, and its impact will be discussed in the relevant subsections. To compare how different configurations—such as thread count, CPU governor, and power mode—affect various performance metrics, we present detailed results for a single model. Phi 3.5 Mini was chosen as the representative model since it is closest to the center of the evaluated parameter range and did not exhibit significant outlier behavior in any metric. While most models followed similar trends across configurations, substantial outliers are highlighted separately. Despite models being similarly affected by configuration changes, their absolute performance levels varied. These differences are analyzed by presenting results for all models under a single selected configuration: *(i)* CPU governor set to *ondemand* (default setting), *(ii)* Power mode 15W (default setting), and *(iii)* Thread count as maximum available, except for generation throughput and energy consumption, where the optimal thread count was used per model and device.

Using the maximum thread count caused significant performance degradation in the generation phase, particularly for *Q4_0* quantization. To address this, we leveraged llama.cpp's capability to assign separate thread counts for the prefill and generation phases, optimizing inference efficiency without negatively impacting system performance. Unlike thread count, modifying CPU governors and power modes affects the entire system, so we opted to keep them at their default settings to maintain consistency across experiments.

5.1 Memory Footprint Analysis

No significant differences in memory usage were observed across different system configurations. Therefore, this subsection focuses on model- and device-level comparisons.

As shown in Figure 3, CPU inference with *Q4_K_M* quantization exhibited an approximately linear relationship between peak memory usage and model size. This behavior is expected, as model weights account for most of the memory footprint. For GPU inference, a larger memory footprint was observed. This

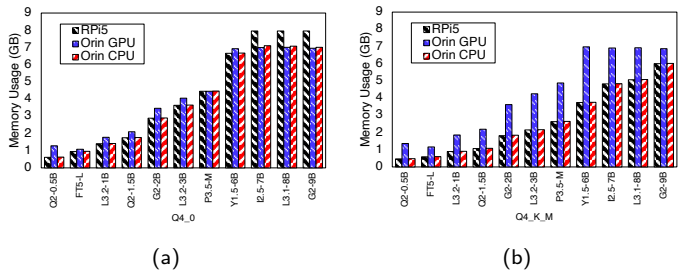

Fig. 3. Peak memory usage for all devices and quantization schemes

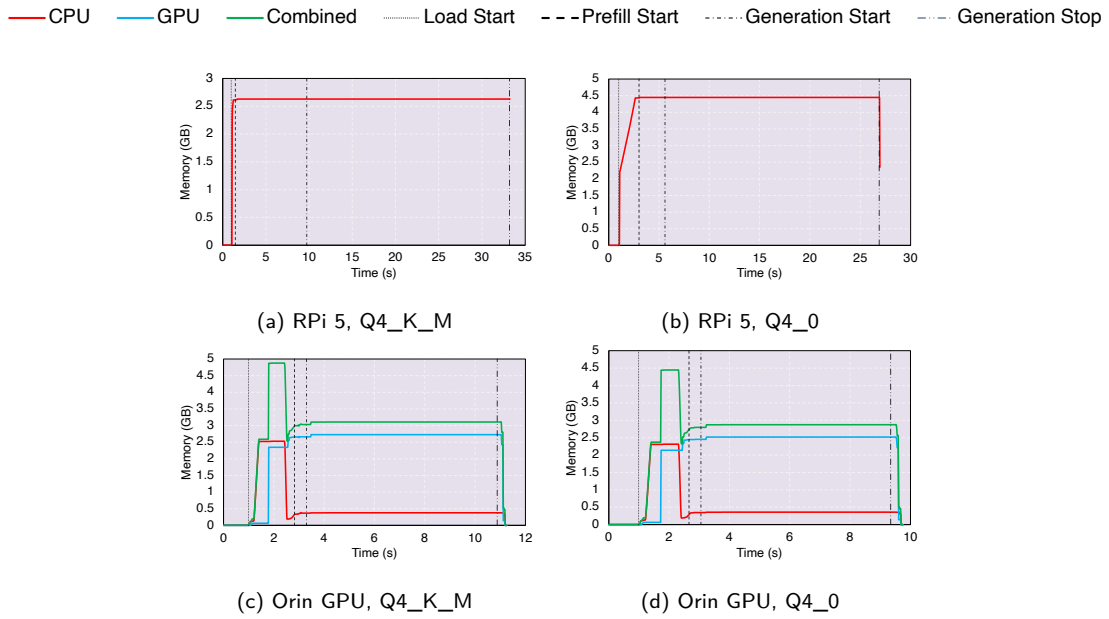

Fig. 4. Memory footprint timeline of one example run for RPi 5 (ondemand, 4 threads), Orin GPU (15 W) and both quantization schemes

is primarily due to the shared main memory between the CPU and GPU on the Orin, leading to additional memory overhead. While the peak GPU memory usage was comparable to CPU inference, the CPU also allocated memory, further increasing overall memory consumption. This effect was particularly pronounced during the load phase, when the model was transferred to the GPU, as seen in Figure 4. During this phase, both GPU and CPU memory usage peaked simultaneously before the CPU memory allocation decreased, but still induced additional overhead. Another notable observation in GPU inference was the flattening of memory usage for models larger than Phi 3.5 Mini. This was due to the device exhausting its available memory. Despite having 8 GB of nominal memory, the usable memory was limited to below 7 GB, as the operating system and background processes collectively occupied over 1 GB.

In contrast, *Q4_0* quantization exhibited several notable differences in memory usage. First, GPU inference with *Q4_0* quantization resulted in a slightly lower peak memory footprint for most models, whereas CPU inference showed a significantly higher memory footprint than *Q4_K_M* quantization, as seen in Figure 3. The increased memory footprint for CPU inference may seem counterintuitive, as *Q4_0*-quantized models are smaller than *Q4_K_M*-quantized models. However, this behavior is likely due to the restructuring of model weights during the load phase to leverage optimized matrix multiplication kernels on the Arm CPU [30]. This restructuring process appears to require additional memory, which remains allocated throughout inference, as observed in Figure 4. This effect is not present in GPU inference, as model weights are not realigned when loaded onto the GPU. Despite this, for most models, GPU inference still had the highest memory footprint. However, for models larger than Yi 1.5 6B, CPU inference with *Q4_0* quantization required more memory than GPU inference. Additionally, for Phi 3.5 Mini, memory usage was nearly identical between CPU and GPU inference. A second key observation is that the flattening of memory usage seen in GPU inference with *Q4_K_M* quantization for larger models was also present in CPU inference with *Q4_0* quantization. As before, this is due to memory exhaustion. The RPi 5's effective memory limit was higher than the Orin's, as its operating system consumed fewer memory resources. However, unlike with *Q4_K_M* quantization, this memory saturation effect did not appear for Yi 1.5 6B under *Q4_0* quantization.

It is worth noting that larger context sizes beyond 512 tokens were also tested. As the context size increases, the memory allocated for the KV-cache grows correspondingly. For extensive context sizes, the KV-cache memory footprint can even exceed the memory required for model weights. Even with a moderate context size of 1028, the largest model began to encounter performance issues. During CPU inference, constant page faults led to sharp performance declines. For GPU inference, the system experienced device crashes and restarts. These findings highlight that memory is a critical bottleneck for use cases requiring larger models or extended context sizes, posing significant challenges for LLM inference on resource-constrained edge devices. Therefore, next-generation edge devices could be equipped with larger memory amounts to be more suitable for memory-intensive GenAI workloads.

Main Takeaways

- Memory usage grows approximately linearly with model size
- GPU inference comes at the cost of significant memory overhead
- *Q4_0* quantization reduces memory footprint for GPU inference but adds substantial memory overhead for CPU inference compared to *Q4_K_M* quantization
- Memory is a main bottleneck for using very large models or context sizes on edge devices.

5.2 Inference Performance: Latency and Throughput

As shown in Figure 5, end-to-end latency generally decreases with an increasing thread count for both the RPi 5 and the Orin CPU under *Q4_K_M* quantization. However, with *Q4_0* quantization, the lowest latency was observed at two threads on the RPi 5 and four threads on the Orin CPU when using the *ondemand* and *performance* governors. This reduction was primarily due to a significantly shorter generation phase. Additionally, the *powersave* governor led to higher latencies compared to *ondemand* and *performance* governors. While no significant difference was observed between *ondemand* and *performance* on the RPi 5, the Orin CPU exhibited slightly lower latencies with the *performance* governor across most thread counts.